

DL_05_redes_para_RL

March 13, 2026

1 Redes Neuronales para Reinforcement Learning

1.1 Introducción

Este notebook explora cómo usar **redes neuronales** en el contexto de **Reinforcement Learning (RL)**. Nos centraremos especialmente en **Deep Q-Networks (DQN)**, el algoritmo que revolucionó el campo al demostrar que agentes podían aprender a jugar videojuegos directamente desde píxeles.

1.1.1 ¿Por qué Deep Learning + RL?

El RL tradicional usa **tablas Q** para almacenar el valor de cada par (estado, acción). Esto funciona bien para problemas pequeños, pero **no escala**:

Problema	Estados posibles	¿Tabla Q viable?
Tic-Tac-Toe	~5,000	Sí
Ajedrez	$\sim 10^{43}$	No
Atari (píxeles)	$256^{(210 \times 160 \times 3)}$	Imposible
Coche autónomo	Continuo	Imposible

La solución: usar una **red neuronal** como **aproximador de función** que generaliza a estados nunca vistos.

1.1.2 Contenido

1. Fundamentos: Q-Learning y DQN
2. Arquitectura de Redes para RL
3. Ejemplo 1: CartPole con DQN
4. Ejemplo 2: Juego de Carreras con DQN
5. Técnicas Avanzadas
6. Resumen y Recursos

```
[1]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
```

```

import matplotlib.pyplot as plt
from collections import deque
import random

# Configuración
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"PyTorch: {torch.__version__}")
print(f"Dispositivo: {DEVICE}")

# Semilla para reproducibilidad
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

```

PyTorch: 2.10.0+cu128

Dispositivo: cpu

1.2 1. Fundamentos: Q-Learning y DQN

1.2.1 1.1 ¿Qué es Q-Learning?

Q-Learning es un algoritmo de RL que aprende una **función $Q(s, a)$** que estima la **recompensa total esperada** al tomar la acción **a** en el estado **s** y seguir la política óptima después.

Ecuación de Bellman (la base teórica):

$$Q(s, a) = r + \gamma \cdot \max_{a'} Q(s', a')$$

Donde: - r = recompensa inmediata - γ = factor de descuento (0.95-0.99) - s' = siguiente estado - $\max_{a'} Q(s', a')$ = mejor valor Q del siguiente estado

1.2.2 1.2 De Q-Table a Deep Q-Network

Q-Table	Deep Q-Network (DQN)
Tabla de valores $Q(s,a)$	Red neuronal que predice $Q(s,a)$
Solo estados discretos	Estados continuos o de alta dimensión
Sin generalización	Generaliza a estados no vistos
Memoria: $O(\text{estados} \times \text{acciones})$	Memoria: $O(\text{parámetros de la red})$

Q-Table:

Estado	Acc0	Acc1	Acc2
S0	0.5	0.2	0.8
S1	0.3	0.9	0.1
...	...	...	...

DQN:

Estado [n] → Red Neuronal → Q-valores [m]
 ↓
 [s0, s1, ...] → [Q(s,a0), Q(s,a1), Q(s,a2)]
 La red APRENDE a mapear estado → Q-valores

1.2.3 1.3 Innovaciones clave de DQN

El paper original de DeepMind (2015) introdujo dos técnicas que hacen viable entrenar redes neuronales con RL:

1. Experience Replay (Replay Buffer) Problema: Las experiencias consecutivas están muy correlacionadas. Si entrenamos secuencialmente, la red “olvida” experiencias pasadas.

Solución: Almacenar experiencias (s, a, r, s', done) en un **buffer** y entrenar con **muestras aleatorias**.

```
# Guardar experiencia
```

```
replay_buffer.append((estado, accion, reward, siguiente_estado, done))
```

```
# Entrenar con batch aleatorio
```

```
batch = random.sample(replay_buffer, batch_size=32)
```

Beneficios: - Rompe correlación temporal - Reutiliza experiencias varias veces - Entrenamiento más estable

2. Target Network (Red Objetivo) Problema: Al calcular el target $r + \gamma \cdot \max Q(s', a')$, usamos la misma red que estamos actualizando. Esto crea inestabilidad (“chasing a moving target”).

Solución: Mantener una **copia congelada** de la red (target network) que se actualiza periódicamente.

```
# Red principal (se actualiza cada paso)
```

```
q_network = DQN()
```

```
# Red objetivo (se actualiza cada N pasos)
```

```
target_network = DQN()
```

```
target_network.load_state_dict(q_network.state_dict())
```

```
# Cada N pasos:
```

```
target_network.load_state_dict(q_network.state_dict())
```

1.2.4 1.4 El algoritmo DQN completo

Inicializar:

- Q-network con pesos aleatorios
- Target network con pesos ' ' =
- Replay buffer vacío D
- $\epsilon = 1.0$ (exploración inicial)

Para cada episodio:

```
estado = env.reset()
```

Para cada paso:

```
# 1. Seleccionar acción (-greedy)
if random() <  $\epsilon$ :
    acción = random_action()
```

```

else:
    acción = argmax(Q(estado)) # La mejor según la red

# 2. Ejecutar acción
siguiente_estado, reward, done = env.step(acción)

# 3. Guardar experiencia
D.append((estado, acción, reward, siguiente_estado, done))

# 4. Entrenar con batch del buffer
batch = sample(D, 32)
for (s, a, r, s', d) in batch:
    if d: # Estado terminal
        target = r
    else:
        target = r + * max(Q_target(s')) # Usar target network

    loss = (Q(s)[a] - target)^2
    optimizer.step(loss)

# 5. Actualizar target network (cada N pasos)
if paso % N == 0:
    ' =

# 6. Decaer
    = max(_min, * decay)

estado = siguiente_estado

```

1.3 2. Arquitectura de Redes para RL

1.3.1 2.1 Diseño de la Red Q

La arquitectura de la red depende del **tipo de observación**:

Tipo de Estado	Arquitectura	Ejemplo
Vector de features	MLP (Fully Connected)	CartPole, sensores
Imágenes	CNN + FC	Atari, juegos visuales
Secuencias	RNN/LSTM + FC	Juegos de memoria

1.3.2 2.2 MLP para estados vectoriales

La arquitectura más simple para estados de baja dimensión (< 100 features):

```

[2]: class DQN_MLP(nn.Module):
    """

```

Red DQN simple para estados vectoriales.

Arquitectura: Estado [n] -> FC(128) -> ReLU -> FC(128) -> ReLU -> Q-valores [m]

Args:

state_size: Dimensión del vector de estado

n_actions: Número de acciones posibles

hidden_size: Neuronas en capas ocultas (default: 128)

"""

```
def __init__(self, state_size, n_actions, hidden_size=128):  
    super().__init__()
```

```
    self.network = nn.Sequential(  
        nn.Linear(state_size, hidden_size),  
        nn.ReLU(),  
        nn.Linear(hidden_size, hidden_size),  
        nn.ReLU(),  
        nn.Linear(hidden_size, n_actions)  
    )
```

```
def forward(self, x):
```

"""

Forward pass.

Args:

x: Estado (batch_size, state_size) o (state_size,)

Returns:

Q-valores para cada acción (batch_size, n_actions)

"""

```
    return self.network(x)
```

Ejemplo de uso

```
print("="*60)
```

```
print("Ejemplo: DQN para CartPole")
```

```
print("="*60)
```

CartPole: estado = [pos_carro, vel_carro, angulo, vel_angular]

Acciones: 0 = empujar izquierda, 1 = empujar derecha

```
dqn_cartpole = DQN_MLP(state_size=4, n_actions=2)
```

```
print(f"\n{dqn_cartpole}")
```

```
print(f"\nParámetros: {sum(p.numel() for p in dqn_cartpole.parameters()),}")
```

```
# Test forward
estado = torch.randn(1, 4) # Un estado de ejemplo
q_valores = dqn_cartpole(estado)
print(f"\nEstado: {estado.squeeze().tolist()}")
print(f"Q-valores: {q_valores.squeeze().tolist()}")
print(f"Mejor acción: {q_valores.argmax().item()}")
```

=====

Ejemplo: DQN para CartPole

=====

```
DQN_MLP(
    (network): Sequential(
      (0): Linear(in_features=4, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=128, bias=True)
      (3): ReLU()
      (4): Linear(in_features=128, out_features=2, bias=True)
    )
)
```

Parámetros: 17,410

Estado: [1.726496934890747, 0.6127423048019409, -1.616715908050537, -1.0682919025421143]

Q-valores: [-0.1496981680393219, 0.03438492864370346]

Mejor acción: 1

1.3.3 2.3 CNN para estados visuales (imágenes)

Para juegos donde el estado es una **imagen** (Atari, videojuegos):

```
[3]: class DQN_CNN(nn.Module):
    """
    Red DQN con CNN para estados visuales (imágenes).

    Arquitectura (estilo Atari DQN original):
        Imágenes [4, 84, 84] -> Conv(32, 8x8, stride 4) -> ReLU
                                -> Conv(64, 4x4, stride 2) -> ReLU
                                -> Conv(64, 3x3, stride 1) -> ReLU
                                -> Flatten -> FC(512) -> ReLU
                                -> FC(n_actions)

    Args:
        n_actions: Número de acciones posibles
        input_shape: Forma de entrada (channels, height, width)
    """
```

```

def __init__(self, n_actions, input_shape=(4, 84, 84)):
    super().__init__()

    c, h, w = input_shape

    # Capas convolucionales
    self.conv = nn.Sequential(
        nn.Conv2d(c, 32, kernel_size=8, stride=4),
        nn.ReLU(),
        nn.Conv2d(32, 64, kernel_size=4, stride=2),
        nn.ReLU(),
        nn.Conv2d(64, 64, kernel_size=3, stride=1),
        nn.ReLU(),
        nn.Flatten()
    )

    # Calcular tamaño de salida de conv
    with torch.no_grad():
        dummy = torch.zeros(1, c, h, w)
        conv_out_size = self.conv(dummy).shape[1]

    # Capas fully connected
    self.fc = nn.Sequential(
        nn.Linear(conv_out_size, 512),
        nn.ReLU(),
        nn.Linear(512, n_actions)
    )

    def forward(self, x):
        """x: Imágenes (batch, channels, height, width)"""
        features = self.conv(x)
        q_values = self.fc(features)
        return q_values

# Ejemplo de uso
print("="*60)
print("Ejemplo: DQN CNN para Atari")
print("="*60)

# Atari: 4 frames apilados de 84x84 (grayscale)
# 18 acciones posibles (típico en Atari)
dqn_atari = DQN_CNN(n_actions=18, input_shape=(4, 84, 84))

print(f"\nParámetros: {sum(p.numel() for p in dqn_atari.parameters()),}")

# Test forward

```

```
frames = torch.randn(1, 4, 84, 84) # 4 frames apilados
q_valores = dqn_atari(frames)
print(f"\nEntrada: {frames.shape}")
print(f"Salida (Q-valores): {q_valores.shape}")
print(f"Mejor acción: {q_valores.argmax().item()}")
```

=====

Ejemplo: DQN CNN para Atari

=====

Parámetros: 1,693,362

Entrada: torch.Size([1, 4, 84, 84])
 Salida (Q-valores): torch.Size([1, 18])
 Mejor acción: 5

1.3.4 2.4 Dueling DQN (Arquitectura Avanzada)

Dueling DQN separa el cálculo de Q en dos partes:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$$

Donde: - **V(s)**: Valor del estado (qué tan bueno es estar aquí) - **A(s, a)**: Ventaja de cada acción (qué tan buena es esta acción vs el promedio)

Beneficio: Aprende el valor del estado incluso sin tomar todas las acciones.

```
[4]: class DuelingDQN(nn.Module):
    """
    Dueling DQN: separa Value y Advantage.

    Arquitectura:
        Estado -> Shared FC -> Value head (1 salida)
                    -> Advantage head (n_actions salidas)

        Q(s,a) = V(s) + (A(s,a) - mean(A(s,:)))
    """

    def __init__(self, state_size, n_actions, hidden_size=128):
        super().__init__()

        # Red compartida
        self.shared = nn.Sequential(
            nn.Linear(state_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU()
```



```

    )

    # Value stream: V(s)
    self.value_head = nn.Sequential(
        nn.Linear(hidden_size, 64),
        nn.ReLU(),
        nn.Linear(64, 1) # Un solo valor
    )

    # Advantage stream: A(s, a)
    self.advantage_head = nn.Sequential(
        nn.Linear(hidden_size, 64),
        nn.ReLU(),
        nn.Linear(64, n_actions)
    )

def forward(self, x):
    shared_features = self.shared(x)

    value = self.value_head(shared_features) # (batch, 1)
    advantage = self.advantage_head(shared_features) # (batch, n_actions)

    #  $Q = V + (A - \text{mean}(A))$ 
    # Restar la media de A para identificabilidad
    q_values = value + (advantage - advantage.mean(dim=1, keepdim=True))

    return q_values

# Comparar arquitecturas
print("="*60)
print("Comparación: DQN vs Dueling DQN")
print("="*60)

dqn_std = DQN_MLP(state_size=4, n_actions=2)
dqn_duel = DuelingDQN(state_size=4, n_actions=2)

print(f"\nDQN estándar:  {sum(p.numel() for p in dqn_std.parameters()):,}␣  

    ↪parámetros")
print(f"Dueling DQN:    {sum(p.numel() for p in dqn_duel.parameters()):,}␣  

    ↪parámetros")

```

```

=====
Comparación: DQN vs Dueling DQN
=====

```

```

DQN estándar:    17,410 parámetros
Dueling DQN:     33,859 parámetros

```

1.4 3. Ejemplo 1: CartPole con DQN

1.4.1 3.1 El problema CartPole

Objetivo: Balancear un palo sobre un carro moviéndolo izquierda/derecha.

```
      |
      | <- Palo (debe mantenerse vertical)
      |
+---+---+
|       | <- Carro
+-----+
<+++++> <- Se puede mover izq/der
```

Estado (4 valores): 1. Posición del carro 2. Velocidad del carro 3. Ángulo del palo 4. Velocidad angular del palo

Acciones (2): - 0: Empujar izquierda - 1: Empujar derecha

Recompensa: +1 por cada paso que el palo sigue en pie.

Fin del episodio: Palo cae ($>\pm 12^\circ$) o carro sale de la pista.

```
[5]: # Primero creamos un entorno CartPole simplificado (sin gymnasium)
      # para mantener el notebook autocontenido

class CartPoleEnv:
    """
    Implementación simplificada de CartPole.

    Física basada en las ecuaciones clásicas del péndulo invertido.
    """

    def __init__(self):
        # Parámetros físicos
        self.gravity = 9.8
        self.cart_mass = 1.0
        self.pole_mass = 0.1
        self.pole_length = 0.5
        self.force_mag = 10.0
        self.dt = 0.02 # Paso de tiempo

        # Límites
        self.x_threshold = 2.4
        self.theta_threshold = 12 * np.pi / 180 # 12 grados en radianes

        # Estado
        self.state = None
        self.steps = 0
```

```

self.max_steps = 500

def reset(self):
    """Reinicia el entorno con estado aleatorio pequeño."""
    self.state = np.random.uniform(-0.05, 0.05, size=4)
    self.steps = 0
    return self.state.copy()

def step(self, action):
    """
    Ejecuta una acción.

    Args:
        action: 0 (izquierda) o 1 (derecha)

    Returns:
        next_state, reward, done, info
    """
    x, x_dot, theta, theta_dot = self.state

    # Fuerza aplicada
    force = self.force_mag if action == 1 else -self.force_mag

    # Física del péndulo invertido
    cos_theta = np.cos(theta)
    sin_theta = np.sin(theta)

    total_mass = self.cart_mass + self.pole_mass
    pole_mass_length = self.pole_mass * self.pole_length

    # Aceleraciones
    temp = (force + pole_mass_length * theta_dot**2 * sin_theta) /
↪total_mass
    theta_acc = (self.gravity * sin_theta - cos_theta * temp) / (
        self.pole_length * (4/3 - self.pole_mass * cos_theta**2 /
↪total_mass)
    )
    x_acc = temp - pole_mass_length * theta_acc * cos_theta / total_mass

    # Integración de Euler
    x = x + self.dt * x_dot
    x_dot = x_dot + self.dt * x_acc
    theta = theta + self.dt * theta_dot
    theta_dot = theta_dot + self.dt * theta_acc

    self.state = np.array([x, x_dot, theta, theta_dot])
    self.steps += 1

```

```

    # Verificar condiciones de terminación
    done = (
        x < -self.x_threshold or x > self.x_threshold or
        theta < -self.theta_threshold or theta > self.theta_threshold or
        self.steps >= self.max_steps
    )

    # Recompensa: +1 por cada paso que sobrevive
    reward = 1.0 if not done else 0.0

    return self.state.copy(), reward, done, {}

# Probar el entorno
print("="*60)
print("Entorno CartPole")
print("="*60)

env = CartPoleEnv()
state = env.reset()
print(f"\nEstado inicial: {state}")
print(f"  [posición, velocidad, ángulo, vel_angular]")

# Ejecutar acción aleatoria
next_state, reward, done, _ = env.step(1) # Empujar derecha
print(f"\nDespués de empujar derecha:")
print(f"  Estado: {next_state}")
print(f"  Reward: {reward}, Done: {done}")

```

```

=====
Entorno CartPole
=====

Estado inicial: [-0.01254599  0.04507143  0.02319939  0.00986585]
  [posición, velocidad, ángulo, vel_angular]

Después de empujar derecha:
  Estado: [-0.01164456  0.23985312  0.02339671 -0.27540805]
  Reward: 1.0, Done: False

```

1.4.2 3.2 Agente DQN completo

```

[6]: class DQNAgent:
    """
    Agente DQN completo con Experience Replay y Target Network.

```

Implementa el algoritmo DQN original de DeepMind.

```
"""  
  
def __init__(self, state_size, n_actions, hidden_size=128):  
    self.state_size = state_size  
    self.n_actions = n_actions  
    self.device = DEVICE  
  
    # Hiperparámetros  
    self.gamma = 0.99          # Factor de descuento  
    self.epsilon = 1.0         # Exploración inicial  
    self.epsilon_min = 0.01    # Exploración mínima  
    self.epsilon_decay = 0.995 # Decaimiento por episodio  
    self.learning_rate = 0.001  
    self.batch_size = 64  
    self.memory_size = 10000  
    self.target_update_freq = 10 # Actualizar target cada N episodios  
  
    # Redes  
    self.q_network = DQN_MLP(state_size, n_actions, hidden_size).to(self.  
↪device)  
    self.target_network = DQN_MLP(state_size, n_actions, hidden_size).  
↪to(self.device)  
    self.target_network.load_state_dict(self.q_network.state_dict())  
    self.target_network.eval() # No entrenar target network  
  
    # Optimizer  
    self.optimizer = optim.Adam(self.q_network.parameters(), lr=self.  
↪learning_rate)  
  
    # Experience Replay Buffer  
    self.memory = deque(maxlen=self.memory_size)  
  
def act(self, state):  
    """  
    Selecciona acción usando política -greedy.  
  
    Con probabilidad : acción aleatoria (exploración)  
    Con probabilidad 1- : mejor acción según Q (explotación)  
    """  
    if random.random() < self.epsilon:  
        return random.randint(0, self.n_actions - 1)  
  
    with torch.no_grad():  
        state_tensor = torch.FloatTensor(state).unsqueeze(0).to(self.device)  
        q_values = self.q_network(state_tensor)  
        return q_values.argmax().item()
```

```

def remember(self, state, action, reward, next_state, done):
    """Guarda experiencia en el replay buffer."""
    self.memory.append((state, action, reward, next_state, done))

def replay(self):
    """
    Entrena la red con un batch del replay buffer.

    Este es el corazón del algoritmo DQN.
    """
    if len(self.memory) < self.batch_size:
        return 0.0

    # Muestrear batch aleatorio
    batch = random.sample(self.memory, self.batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)

    # Convertir a tensores
    states = torch.FloatTensor(np.array(states)).to(self.device)
    actions = torch.LongTensor(actions).to(self.device)
    rewards = torch.FloatTensor(rewards).to(self.device)
    next_states = torch.FloatTensor(np.array(next_states)).to(self.device)
    dones = torch.FloatTensor(dones).to(self.device)

    # Q-valores actuales para las acciones tomadas
    current_q = self.q_network(states).gather(1, actions.unsqueeze(1)).
    ↪squeeze()

    # Q-valores objetivo (usando target network)
    with torch.no_grad():
        next_q = self.target_network(next_states).max(1)[0]
        target_q = rewards + (1 - dones) * self.gamma * next_q

    # Loss y backpropagation
    loss = F.mse_loss(current_q, target_q)

    self.optimizer.zero_grad()
    loss.backward()

    # Gradient clipping para estabilidad
    nn.utils.clip_grad_norm_(self.q_network.parameters(), max_norm=1.0)
    self.optimizer.step()

    return loss.item()

def update_target(self):
    """Copia pesos de q_network a target_network."""

```

```

        self.target_network.load_state_dict(self.q_network.state_dict())

    def decay_epsilon(self):
        """Reduce exploración gradualmente."""
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

print("Agente DQN creado:")
agent = DQNAgent(state_size=4, n_actions=2)
print(f"  Red Q: {sum(p.numel() for p in agent.q_network.parameters()),}␣
      ↪parámetros")
print(f"  Memoria: {agent.memory_size:,} experiencias")
print(f"  inicial: {agent.epsilon}")

```

Agente DQN creado:

Red Q: 17,410 parámetros

Memoria: 10,000 experiencias

inicial: 1.0

1.4.3 3.3 Entrenamiento

```

[7]: def train_dqn(env, agent, episodes=200, verbose=True):
    """
    Entrena un agente DQN en un entorno.

    Args:
        env: Entorno (debe tener reset() y step())
        agent: Agente DQN
        episodes: Número de episodios
        verbose: Mostrar progreso

    Returns:
        Lista de recompensas por episodio
    """
    rewards_history = []

    for episode in range(episodes):
        state = env.reset()
        total_reward = 0
        done = False

        while not done:
            # 1. Seleccionar acción
            action = agent.act(state)

            # 2. Ejecutar acción
            next_state, reward, done, _ = env.step(action)

```

```

    # 3. Guardar experiencia
    agent.remember(state, action, reward, next_state, done)

    # 4. Entrenar
    agent.replay()

    state = next_state
    total_reward += reward

    # Fin del episodio
    rewards_history.append(total_reward)
    agent.decay_epsilon()

    # Actualizar target network
    if episode % agent.target_update_freq == 0:
        agent.update_target()

    # Log
    if verbose and (episode + 1) % 20 == 0:
        avg_reward = np.mean(rewards_history[-20:])
        print(f"Ep {episode+1:3d} | Reward: {total_reward:3.0f} | "
              f"Avg(20): {avg_reward:5.1f} | : {agent.epsilon:.3f}")

    return rewards_history

# Entrenar
print("="*60)
print("ENTRENAMIENTO DQN en CartPole")
print("="*60 + "\n")

env = CartPoleEnv()
agent = DQNAgent(state_size=4, n_actions=2)

rewards = train_dqn(env, agent, episodes=150)

```

```

=====
ENTRENAMIENTO DQN en CartPole
=====

```

```

Ep  20 | Reward:  18 | Avg(20):  20.2 | : 0.905
Ep  40 | Reward:  13 | Avg(20):  21.6 | : 0.818
Ep  60 | Reward:  19 | Avg(20):  28.1 | : 0.740
Ep  80 | Reward:  21 | Avg(20):  45.5 | : 0.670
Ep 100 | Reward:  69 | Avg(20):  35.8 | : 0.606
Ep 120 | Reward:  46 | Avg(20):  65.0 | : 0.548
Ep 140 | Reward: 126 | Avg(20):  72.0 | : 0.496

```



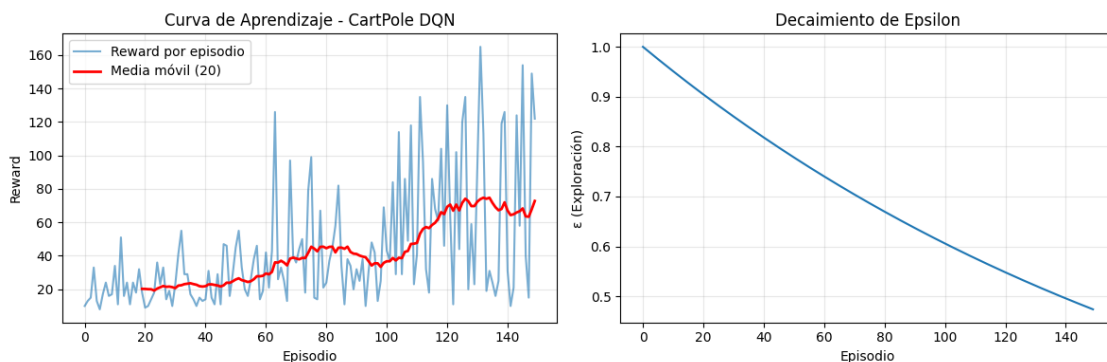
```
[8]: # Visualizar resultados
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(rewards, alpha=0.6, label='Reward por episodio')
# Media móvil
window = 20
avg_rewards = np.convolve(rewards, np.ones(window)/window, mode='valid')
plt.plot(range(window-1, len(rewards)), avg_rewards, 'r-', linewidth=2,
        label=f'Media móvil ({window})')
plt.xlabel('Episodio')
plt.ylabel('Reward')
plt.title('Curva de Aprendizaje - CartPole DQN')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
# Epsilon decay
epsilons = [1.0]
for _ in range(len(rewards)-1):
    epsilons.append(max(0.01, epsilons[-1] * 0.995))
plt.plot(epsilons)
plt.xlabel('Episodio')
plt.ylabel('ε (Exploración)')
plt.title('Decaimiento de Epsilon')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\nReward máximo: {max(rewards):.0f}")
print(f"Reward promedio (últimos 20): {np.mean(rewards[-20:]):.1f}")
```



Reward máximo: 165

Reward promedio (últimos 20): 72.8

1.5 4. Ejemplo 2: Juego de Carreras con DQN

Este ejemplo muestra cómo aplicar DQN a un juego más complejo: coches que aprenden a conducir en un circuito.

Características del problema: - **Estado:** 6 valores (5 sensores de distancia + velocidad) - **Acciones:** 9 combinaciones de acelerar/frenar + girar izq/der - **Multi-agente:** 4 coches compitiendo

1.5.1 4.1 Arquitectura de la red para el coche

```
[9]: class CarDQN(nn.Module):
    """
    Red DQN para un coche de carreras.

    Estado (6 dims):
    [sensor_izq_60, sensor_izq_30, sensor_frente,
     sensor_der_30, sensor_der_60, velocidad]

    Acciones (9):
    0: Nada
    1: Acelerar
    2: Frenar
    3: Izquierda
    4: Derecha
    5: Acelerar + Izquierda
    6: Acelerar + Derecha
    7: Frenar + Izquierda
    8: Frenar + Derecha
    """

    def __init__(self, state_size=6, n_actions=9):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, n_actions)
        )

    def forward(self, x):
        return self.network(x)
```

```

# Mostrar arquitectura
print("="*60)
print("Red DQN para Coche de Carreras")
print("="*60)

car_dqn = CarDQN()
print(f"\n{car_dqn}")
print(f"\nParámetros: {sum(p.numel() for p in car_dqn.parameters()):,}")

# Simular sensores
estado_coche = torch.tensor([
    0.8,  # Sensor izq 60° (lejos del borde)
    0.6,  # Sensor izq 30°
    0.3,  # Sensor frente (cerca del borde!)
    0.7,  # Sensor der 30°
    0.9,  # Sensor der 60° (muy lejos)
    0.5   # Velocidad normalizada
]).unsqueeze(0)

q_valores = car_dqn(estado_coche)
mejor_accion = q_valores.argmax().item()

acciones = ['Nada', 'Acelerar', 'Frenar', 'Izq', 'Der',
            'Acel+Izq', 'Acel+Der', 'Fren+Izq', 'Fren+Der']

print(f"\nEstado de sensores:")
print(f"  Izq 60°: {estado_coche[0,0]:.1f} | Izq 30°: {estado_coche[0,1]:.1f}")
print(f"  Frente: {estado_coche[0,2]:.1f} (CERCA!)")
print(f"  Der 30°: {estado_coche[0,3]:.1f} | Der 60°: {estado_coche[0,4]:.1f}")
print(f"  Velocidad: {estado_coche[0,5]:.1f}")
print(f"\nMejor acción según Q: {acciones[mejor_accion]} (Q={q_valores[0,mejor_accion]:.3f})")

```

```

=====
Red DQN para Coche de Carreras
=====

```

```

CarDQN(
  (network): Sequential(
    (0): Linear(in_features=6, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=9, bias=True)
  )
)

```

Parámetros: 18,569

Estado de sensores:

Izq 60°: 0.8 | Izq 30°: 0.6

Frente: 0.3 (CERCA!)

Der 30°: 0.7 | Der 60°: 0.9

Velocidad: 0.5

Mejor acción según Q: Acel+Izq (Q=0.173)

1.5.2 4.2 Agente para el coche

```
[10]: class CarAgent:
    """
    Agente DQN para un coche de carreras.

    Cada coche tiene su propia red y memoria (agentes independientes).
    """

    def __init__(self, agent_id, state_size=6, n_actions=9):
        self.id = agent_id
        self.state_size = state_size
        self.n_actions = n_actions
        self.device = DEVICE

        # Hiperparámetros
        self.gamma = 0.95
        self.epsilon = 1.0
        self.epsilon_min = 0.05
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.batch_size = 32

        # Redes
        self.q_network = CarDQN(state_size, n_actions).to(self.device)
        self.target_network = CarDQN(state_size, n_actions).to(self.device)
        self.target_network.load_state_dict(self.q_network.state_dict())

        self.optimizer = optim.Adam(self.q_network.parameters(), lr=self.
↪learning_rate)
        self.memory = deque(maxlen=50000)

    def act(self, state):
        """-greedy action selection."""
        if random.random() < self.epsilon:
            return random.randint(0, self.n_actions - 1)
```

```

        with torch.no_grad():
            state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
            q_values = self.q_network(state_t)
            return q_values.argmax().item()

def remember(self, state, action, reward, next_state, done):
    self.memory.append((state, action, reward, next_state, done))

def replay(self):
    if len(self.memory) < self.batch_size:
        return 0.0

    batch = random.sample(self.memory, self.batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)

    states = torch.FloatTensor(np.array(states)).to(self.device)
    actions = torch.LongTensor(actions).to(self.device)
    rewards = torch.FloatTensor(rewards).to(self.device)
    next_states = torch.FloatTensor(np.array(next_states)).to(self.device)
    dones = torch.FloatTensor(dones).to(self.device)

    current_q = self.q_network(states).gather(1, actions.unsqueeze(1)).
    ↪squeeze()

    with torch.no_grad():
        next_q = self.target_network(next_states).max(1)[0]
        target_q = rewards + (1 - dones) * self.gamma * next_q

    loss = F.mse_loss(current_q, target_q)

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    return loss.item()

def update_target(self):
    self.target_network.load_state_dict(self.q_network.state_dict())

def decay_epsilon(self):
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

print("Agentes de coches creados:")
agents = [CarAgent(i) for i in range(4)]
print(f" 4 agentes independientes")

```

```
print(f"  Parámetros totales: {sum(p.numel() for p in agents[0].q_network.
↳parameters()) * 4 * 2:,}")
print(f"  (4 agentes × 2 redes cada uno)")
```

Agentes de coches creados:

4 agentes independientes

Parámetros totales: 148,552

(4 agentes × 2 redes cada uno)

1.5.3 4.3 Función de recompensa

El diseño de la recompensa es **crucial** en RL. Aquí usamos:

Evento	Recompensa	Propósito
Avanzar	$+0.1 \times \Delta \text{distancia}$	Incentivar movimiento
Velocidad > 0	+0.1	Evitar quedarse quieto
Colisión	-10.0	Penalizar choques

```
[11]: def calculate_reward(car_alive, distance_delta, speed):
    """
    Calcula la recompensa para un coche.

    Args:
        car_alive: Si el coche sigue vivo
        distance_delta: Cambio en distancia recorrida
        speed: Velocidad actual del coche

    Returns:
        Recompensa escalar
    """
    if not car_alive:
        return -10.0 # Penalización por colisión

    reward = distance_delta * 0.1 # Recompensa por avanzar

    if speed > 0:
        reward += 0.1 # Bonus por moverse

    return reward

# Ejemplos de recompensas
print("Ejemplos de recompensas:")
print(f"  Avanzar 10 unidades a velocidad > 0: {calculate_reward(True, 10, 5):.
↳1f}")
print(f"  Quedarse quieto: {calculate_reward(True, 0, 0):.1f}")
print(f"  Colisionar: {calculate_reward(False, 0, 0):.1f}")
```

Ejemplos de recompensas:

Avanzar 10 unidades a velocidad > 0: 1.1

Quedarse quieto: 0.0

Colisionar: -10.0

1.5.4 4.4 Variantes de arquitectura multi-agente

El proyecto de carreras explora varias formas de organizar múltiples agentes:

```
[12]: # Variante: Red Compartida
class SharedCarDQN(nn.Module):
    """
    DQN con encoder compartido y cabezas por coche.

    Todos los coches comparten las capas iniciales (aprenden
    representaciones comunes) pero tienen cabezas separadas
    para especializarse.

    Ventaja: Menos parámetros, transfer learning implícito.
    """

    def __init__(self, state_size=6, n_actions=9, n_cars=4):
        super().__init__()
        self.n_cars = n_cars

        # Encoder compartido
        self.shared_encoder = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU()
        )

        # Cabezas independientes
        self.heads = nn.ModuleList([
            nn.Linear(128, n_actions) for _ in range(n_cars)
        ])

    def forward(self, x, car_id):
        """Forward para un coche específico."""
        features = self.shared_encoder(x)
        return self.heads[car_id](features)

# Comparar parámetros
independent = [CarDQN() for _ in range(4)]
shared = SharedCarDQN(n_cars=4)
```

```

params_independent = sum(sum(p.numel() for p in net.parameters()) for net in_
↳independent)
params_shared = sum(p.numel() for p in shared.parameters())

print("="*60)
print("Comparación: Agentes Independientes vs Red Compartida")
print("="*60)
print(f"\n4 redes independientes: {params_independent:,} parámetros")
print(f"1 red compartida:          {params_shared:,} parámetros")
print(f"Reducción:                  {(1 - params_shared/params_independent)*100:.
↳1f}%")

```

```

=====
Comparación: Agentes Independientes vs Red Compartida
=====

```

```

4 redes independientes: 74,276 parámetros
1 red compartida:      22,052 parámetros
Reducción:             70.3%

```

1.6 5. Técnicas Avanzadas

1.6.1 5.1 Double DQN

Problema: DQN tiende a **sobreestimar** los Q-valores porque usa el mismo valor para seleccionar y evaluar la acción.

Solución: Usar la red principal para **seleccionar** la mejor acción, pero la target network para **evaluar** su valor.

```

[13]: def double_dqn_target(q_network, target_network, next_states, rewards, done,
↳gamma):
    """
    Calcula target Q-values usando Double DQN.

    DQN estándar:
        target = r + γ * max_a' Q_target(s', a')

    Double DQN:
        a* = argmax_a' Q(s', a')           # Selección con red principal
        target = r + γ * Q_target(s', a*)  # Evaluación con target
    """
    with torch.no_grad():
        # Red principal selecciona la mejor acción
        best_actions = q_network(next_states).argmax(dim=1, keepdim=True)

        # Target network evalúa esa acción

```



```

        next_q = target_network(next_states).gather(1, best_actions).squeeze()

        target = rewards + (1 - dones) * gamma * next_q

    return target

print("Double DQN:")
print("  1. Red principal elige: a* = argmax Q(s', a')")
print("  2. Target evalúa: Q_target(s', a*)")
print("  -> Reduce sobreestimación de Q-valores")

```

Double DQN:

1. Red principal elige: $a^* = \operatorname{argmax} Q(s', a')$
2. Target evalúa: $Q_{\text{target}}(s', a^*)$
- > Reduce sobreestimación de Q-valores

1.6.2 5.2 Prioritized Experience Replay

Idea: No todas las experiencias son igual de útiles. Muestrear con **mayor probabilidad** las experiencias con **mayor error TD** (las que más “sorprenden” a la red).

```

[14]: class PrioritizedReplayBuffer:
    """
    Replay buffer con muestreo prioritario.

    Experiencias con mayor error TD tienen mayor probabilidad
    de ser seleccionadas para entrenamiento.
    """

    def __init__(self, capacity, alpha=0.6):
        """
        Args:
            capacity: Tamaño máximo del buffer
            alpha: Cuánto priorizar (0 = uniforme, 1 = solo prioridad)
        """
        self.capacity = capacity
        self.alpha = alpha
        self.buffer = []
        self.priorities = np.zeros(capacity, dtype=np.float32)
        self.position = 0

    def add(self, experience, td_error=None):
        """Añade experiencia con prioridad inicial alta."""
        max_priority = self.priorities.max() if self.buffer else 1.0

        if len(self.buffer) < self.capacity:
            self.buffer.append(experience)
        else:

```

```

        self.buffer[self.position] = experience

        self.priorities[self.position] = max_priority
        self.position = (self.position + 1) % self.capacity

def sample(self, batch_size, beta=0.4):
    """
    Muestrea batch según prioridades.

    Args:
        batch_size: Tamaño del batch
        beta: Corrección de importance sampling

    Returns:
        experiences, indices, weights
    """
    n = len(self.buffer)
    priorities = self.priorities[:n] ** self.alpha
    probabilities = priorities / priorities.sum()

    indices = np.random.choice(n, batch_size, p=probabilities,
↪replace=False)
    experiences = [self.buffer[i] for i in indices]

    # Importance sampling weights
    weights = (n * probabilities[indices]) ** (-beta)
    weights /= weights.max() # Normalizar

    return experiences, indices, weights

def update_priorities(self, indices, td_errors):
    """Actualiza prioridades basadas en nuevos TD errors."""
    for idx, td_error in zip(indices, td_errors):
        self.priorities[idx] = abs(td_error) + 1e-6 # Evitar 0

print("Prioritized Experience Replay:")
print(" - Muestrea más experiencias 'sorprendentes'")
print(" - TD error alto -> mayor prioridad")
print(" - Usa importance sampling para corregir sesgo")

```

Prioritized Experience Replay:

- Muestrea más experiencias 'sorprendentes'
- TD error alto -> mayor prioridad
- Usa importance sampling para corregir sesgo

1.6.3 5.3 Resumen de mejoras de DQN

Técnica	Problema que resuelve	Implementación
Experience Replay	Correlación temporal	Buffer + muestreo aleatorio
Target Network	Objetivos móviles	Red congelada actualizada periódicamente
Double DQN	Sobreestimación Q	Separar selección y evaluación
Dueling DQN	Ineficiencia	Separar $V(s)$ y $A(s,a)$
Prioritized Replay	Muestreo ineficiente	Priorizar por TD error
N-step Returns	Bootstrap lento	Usar recompensas de N pasos

Rainbow DQN combina todas estas técnicas y es actualmente el estado del arte.

1.7 6. Resumen y Recursos

1.7.1 Lo que aprendimos

1. **DQN** usa redes neuronales para aproximar la función Q
2. **Experience Replay** y **Target Network** estabilizan el entrenamiento
3. La **arquitectura** depende del tipo de estado (MLP, CNN, etc.)
4. El **diseño de recompensas** es crítico para el aprendizaje
5. Hay muchas **mejoras** sobre DQN básico (Double, Dueling, PER...)

1.7.2 Código del proyecto Racing

El proyecto completo de carreras con DQN está en:

```
Reinforcement Learning/03_proyectos_dqn/racing/
    racing_game.py      # Juego + agentes DQN
    racing_rl.ipynb     # Análisis detallado
```

1.7.3 Recursos adicionales

Papers: - [Playing Atari with Deep RL](#) - DQN original - [Human-level control through deep RL](#) - DQN Nature - [Rainbow: Combining Improvements](#) - Estado del arte

Tutoriales: - [PyTorch DQN Tutorial](#) - [Spinning Up in Deep RL](#) - OpenAI

Siguiente: Arquitecturas avanzadas (Policy Gradient, A2C, PPO)